

Started on	Tuesday, 5 August 2025, 9:19 PM
State	Finished
Completed on	Tuesday, 5 August 2025, 9:59 PM
Time taken	40 mins 21 secs
Grade	6.69 out of 10.00 (67%)

Question 1

Correct

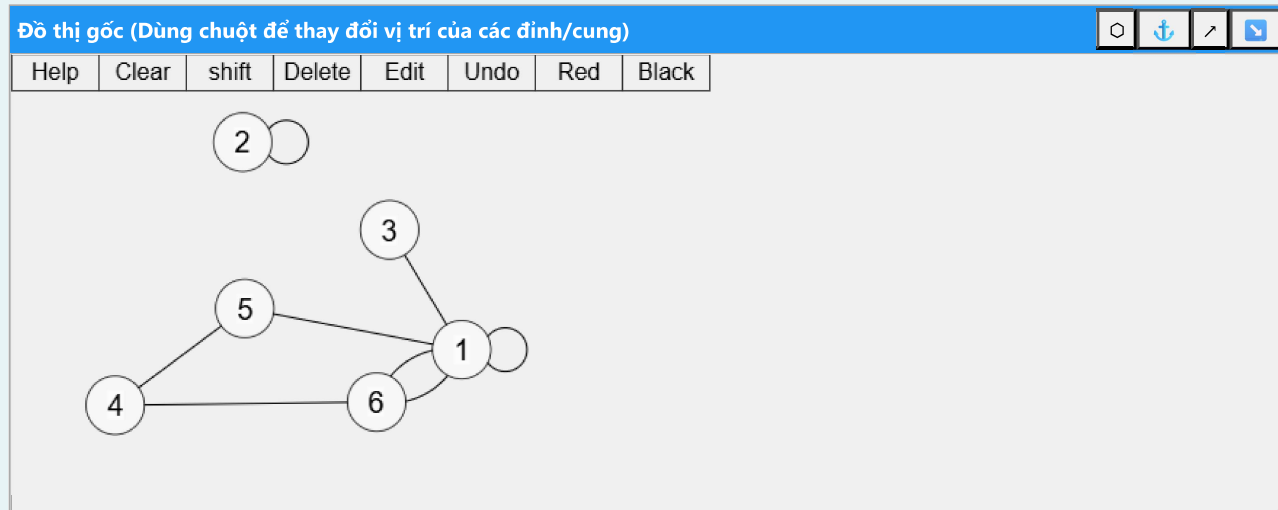
Mark 0.90 out of 1.00

Cho đồ thị **vô hướng** gồm **6** đỉnh và **8** cung như hình vẽ.

Hãy hoàn thành các bài tập bên dưới.

Answer: (penalty regime: 75, 100, ... %)

Reset answer



1. Loại đồ thị (Graph type)

Đồ thị trên thuộc loại nào?

- ☐ Đơn đồ thị (Simple graph)
- ☐ Đa đồ thị (Multigraph)
- ☒ Giả đồ thị (Pseudograph)

2. Kề nhau (adjacent)

Hai đỉnh u và v được gọi là **kề nhau (adjacent)** $\Leftrightarrow$ có cung (u, v) .

Đánh dấu $\checkmark$ vào các câu đúng, bỏ trống các câu sai.

- ☒ 1 và 1 kề nhau.
- ☐ 6 và 6 kề nhau.
- ☐ 4 và 1 kề nhau.

3. Đỉnh kề/lân cận (adjacent vertices or neighbors)

Đỉnh v là **đỉnh kề/lân cận (adjacent vertex/neighbor)** của đỉnh $u \Leftrightarrow u$ và v kề nhau.

Liệt kê các đỉnh kề của các đỉnh bên dưới, ngăn cách nhau bằng dấu phẩy, bỏ qua các đỉnh trùng nhau.

- Các đỉnh kề của 1: 1,3,5,6
- Các đỉnh kề của 2: 2
- Các đỉnh kề của 6: 1,4

4. Bậc (degree)

Bậc của đỉnh u , kí hiệu $\deg(u)$, là số cung liên thuộc với u (khuyến được tính 2 lần).

Cho biết bậc của các đỉnh bên dưới.

- $\deg(3) =$ 1
- $\deg(1) =$ 6
- $\deg(4) =$ 2

	Test	Got	
✓	Test tự động	1. Loại đồ thị - Ok: Giả đồ thị (pseudograph) 2. Kề nhau - 1 và 1: ok. - 6 và 6: ok. - 4 và 1: ok. 3. Đỉnh kề - Các đỉnh kề của 1: ok. - Các đỉnh kề của 2: ok. - Các đỉnh kề của 6: ok. 4. Bậc - Bậc của 3: ok. - Bậc của 1: ok. - Bậc của 4: ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.90/1.00**.

Question **2**

Correct

Mark 1.00 out of 1.00

Cho đồ thị **vô hướng, có trọng số** gồm **5** đỉnh và **6** cung như hình vẽ.

Hãy biểu diễn đồ thị trên bằng phương pháp "**Danh sách cung**" (edge list).

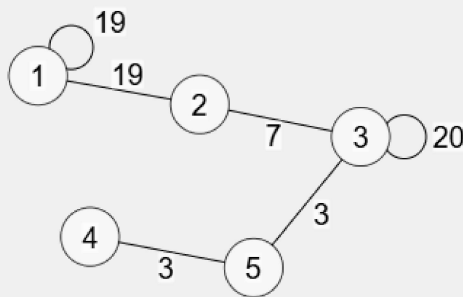
- Liệt kê các cung và trọng số của nó.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc (Dùng chuột để thay đổi vị trí của các đỉnh/cung)

Help Clear shift Delete Edit Undo Red Black



Danh sách các cung (u, v, w), kèm theo trọng số, có trong đồ thị trên

1. (1 , 1 , 19)
2. (1 , 2 , 19)
3. (2 , 3 , 7)
4. (3 , 3 , 20)
5. (3 , 5 , 3)
6. (4 , 5 , 3)

	Test	Got	
✓	Test tự động	Danh sách cung - Cung 1: ok. - Cung 2: ok. - Cung 3: ok. - Cung 4: ok. - Cung 5: ok. - Cung 6: ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 3

Correct

Mark 1.00 out of 1.00

Vẽ đồ thị **vô hướng** có 6 đỉnh và danh sách đỉnh kề của các đỉnh như sau:

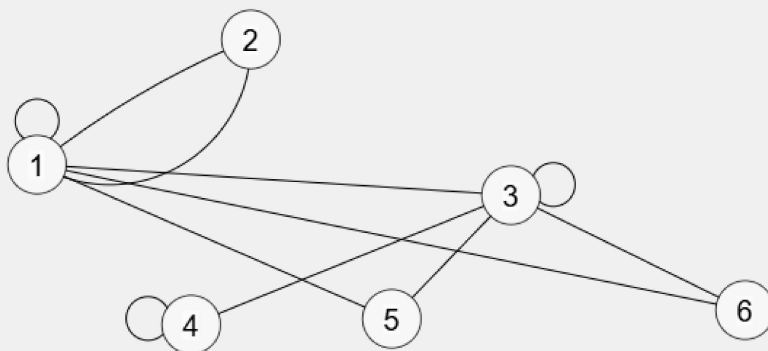
```
1: 1 2 2 3 5 6
2: 1 1
3: 1 3 4 5 6
4: 3 4
5: 1 3
6: 1 3
```

Chú ý

- Đồ thị có thể có đa cung.
- Đồ thị có thể có khuyên.

Answer: (penalty regime: 10, 20, ... %)

Help Clear shift Delete Edit Undo Red Black



	Test	Expected	Got	
✓	#test	1: 1 2 2 3 5 6 2: 1 1 3: 1 3 4 5 6 4: 3 4 5: 1 3 6: 1 3	1: 1 2 2 3 5 6 2: 1 1 3: 1 3 4 5 6 4: 3 4 5: 1 3 6: 1 3	✓

Passed all tests! ✓

Question author's solution (Python3):

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Correct

Marks for this submission: 1.00/1.00.

Question 4

Correct

Mark 1.00 out of 1.00

Xét 1 thuật toán đệ quy duyệt đồ thị theo chiều sâu (**kiểm tra ngoài vòng lặp**) như sau:

```
void DFS(int u) {
    //1. Xét u đã duyệt chưa
    if (mark[u] == 1) //nếu u đã duyệt
        return; //1a. Bỏ qua

    mark[u] = 1; //1b. Duyệt u và

    //2. Xét các đỉnh kề chưa duyệt của u
    for (v là các đỉnh kề chưa duyệt của u)
        DFS(v); //Gọi đệ quy duyệt v
}
```

Thuật toán trên gồm 2 bước chính:

1. Xét u đã duyệt hay chưa? Nếu đã duyệt, bấm "**1a. Bỏ qua**". Ngược lại, bấm "**1b. Duyệt**".
2. Nếu ở bước 1 chọn "**1b. Duyệt**" thì làm tiếp bước 2, nếu không bỏ qua bước 2 (vì đã return). Ở bước 2, lần lượt xét từng đỉnh kề chưa được duyệt của u để gọi đệ quy duyệt nó.
 - Hãy liệt kê các đỉnh kề **chưa được duyệt** của u vào ô "**Với các đỉnh kề chưa duyệt của u**", ngăn cách nhau bằng dấu phẩy. Nếu u không có đỉnh kề hoặc các đỉnh kề của u đều đã được duyệt thì để trống.
 - Bấm nút "**Xét**".

Cho đồ thị **có hướng** gồm **5** đỉnh và **7** cung như bên như bên dưới. Hãy áp dụng thuật toán đệ quy trên để duyệt đồ thị đã cho bắt đầu từ đỉnh **B**.

DFS(B);

Dựa vào kết quả duyệt đồ thị, vẽ cây duyệt đồ thị theo chiều sâu. Cây duyệt đồ thị bao gồm tất cả cá đỉnh của đồ thị gốc và các cung (u, v) với u là đỉnh gọi DFS(v).

Quy ước

- Mỗi thể hiện của hàm **DFS(u)** được minh họa bằng một khung chữ nhật. Việc gọi đệ quy được minh họa bằng một hình chữ nhật nhỏ hơn bên trong.
- Hãy làm bài theo thứ tự từ ngoài vào trong và từ trên xuống dưới.
- Nếu làm sai 1 bước, có thể bấm "**Lùi lại 1 bước**" để làm lại bước trước đó.
- Liệt kê các đỉnh theo thứ tự **A, B, C, ...**

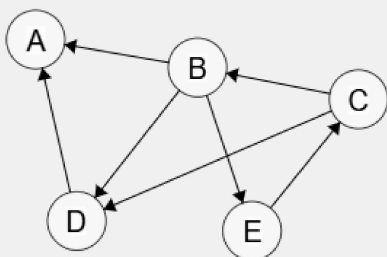
Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc (Dùng chuột để thay đổi vị trí của các đỉnh/cung)



Help Clear shift Delete Edit Undo Red Black



Thực hiện duyệt đệ quy theo chiều sâu bắt đầu từ đỉnh B

🔁 Lùi lại 1 bước Số bước: 10

DFS(B)

1. Nếu B đã duyệt => Bỏ qua, ngược lại => Duyệt

1a. Bỏ qua

1b. Duyệt



1

2. Với các đỉnh kề chưa duyệt của B: A,D,E

Xét



2

DFS(A)

1. Nếu A đã duyệt => Bỏ qua, ngược lại => Duyệt

1a. Bỏ qua

1b. Duyệt



3

2. Với các đỉnh kề chưa duyệt của A:

Xét



4

DFS(D)

1. Nếu D đã duyệt => Bỏ qua, ngược lại => Duyệt

1a. Bỏ qua

1b. Duyệt



5

2. Với các đỉnh kề chưa duyệt của D:

Xét



6

DFS(E)

1. Nếu E đã duyệt => Bỏ qua, ngược lại => Duyệt

1a. Bỏ qua

1b. Duyệt



7

2. Với các đỉnh kề chưa duyệt của E: C

Xét



8

DFS(C)

1. Nếu C đã duyệt => Bỏ qua, ngược lại => Duyệt

1a. Bỏ qua

1b. Duyệt



9

2. Với các đỉnh kề chưa duyệt của C:

Xét



10

Vẽ cây duyệt đồ thị theo chiều sâu

Help

Clear

shift

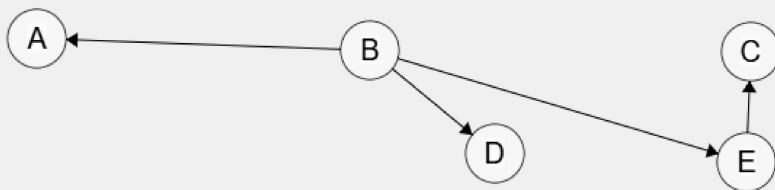
Delete

Edit

Undo

Red

Black



	Test	Got	
✓	Test tự động	1. Kiểm tra DFS - Bước 1. DFS(B): B 1b. Duyệt - Bước 2. DFS(B): B 2. Xét các đỉnh kề chưa duyệt của - Bước 3. DFS(A): A 1b. Duyệt - Bước 4. DFS(A): A 2. Xét các đỉnh kề chưa duyệt của - Bước 5. DFS(D): D 1b. Duyệt - Bước 6. DFS(D): D 2. Xét các đỉnh kề chưa duyệt của - Bước 7. DFS(E): E 1b. Duyệt - Bước 8. DFS(E): E 2. Xét các đỉnh kề chưa duyệt của - Bước 9. DFS(C): C 1b. Duyệt - Bước 10. DFS(C): C 2. Xét các đỉnh kề chưa duyệt của 2. Kiểm tra cây DFS - Xét đỉnh A: Ok. - Xét đỉnh B: Ok. - Xét đỉnh C: Ok. - Xét đỉnh D: Ok. - Xét đỉnh E: Ok. Cây duyệt theo chiều sâu Ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 0.90 out of 1.00

Thuật toán Tarjan tìm các thành phần liên thông mạnh (Tarjan's strongly connected components algorithm) dựa trên thuật toán duyệt đồ thị theo chiều sâu (sử dụng đệ quy) kết hợp với đánh số các đỉnh và một ngăn xếp lưu trữ các thành phần liên thông mạnh (strongly connected components gọi tắt là SCC).

```
void SCC(int u) {
    //1. Đánh số cho đỉnh u, đưa u vào ngăn xếp
    num[u] = k; min_num[u] = k; k++;
    push(S, u); on_stack[u] = true;

    //2. Lần lượt xét các đỉnh kề của u
    for (v là các đỉnh kề của u)
        if (v chưa duyệt) {
            //2a. Duyệt v và cập nhật lại min_num[u]
            SCC(v);
            min_num[u] = min(min_num[u], min_num[v]);
        } else if (v còn trên stack) {
            //2b. cập nhật lại min_num[u]
            min_num[u] = min(min_num[u], num[v]);
        } else {
            //2c. bỏ qua, không làm gì cả
        }

    //3. Kiểm tra num[u] == min_num[u]
    if (num[u] == min_num[u]) {
        //Lấy các đỉnh trong stack ra cho đến khi gặp u
        //Các đỉnh này thuộc về một thành phần liên thông
    }
}
```

Thuật toán trên gồm 3 bước chính:

1. Đánh số cho đỉnh u, đưa u vào ngăn xếp. Sau khi đánh số xong tăng k lên. Thông thường, k được khởi tạo bằng 1.
2. Xét các đỉnh kề v của u, có 3 trường hợp xảy ra:
 - v chưa duyệt => gọi đệ quy duyệt v và cập nhật lại min_num[u]
 - v duyệt rồi nhưng vẫn còn trên stack => cập nhật lại min_num[u]
 - v duyệt rồi và không còn trên stack => bỏ qua
3. Kiểm tra num[u] và min_num[u]: nếu bằng nhau thì u là đỉnh khớp (articulation vertex) hay đỉnh cắt (cut vertex)
 - Lần lượt lấy các đỉnh trong stack ra cho đến khi u được lấy ra. Các đỉnh được lấy ra này chính là thành phần liên thông mạnh chứa u.

Cho đồ thị **có hướng** gồm **5** đỉnh và **6** cung như bên như bên dưới. Hãy áp dụng thuật toán Tarjan để tìm các thành phần liên thông mạnh của G bắt đầu từ đỉnh **C**.

Dựa vào kết quả của thuật toán, vẽ các thành phần liên thông tìm được. Với mỗi thành phần liên thông, vẽ các đỉnh và các cung bên trong thành phần liên thông này. Nói cách khác, xóa bỏ các cung của đồ thị gốc mà 2 đỉnh của nó nằm ở 2 thành phần liên thông khác nhau.

Quy ước

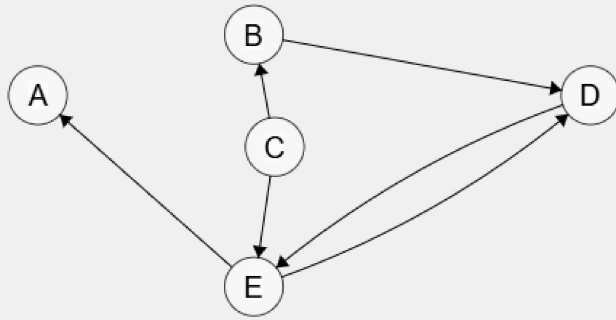
- Mỗi thể hiện của hàm **SCC(u)** được minh họa bằng một khung chữ nhật. Việc gọi đệ quy được minh họa bằng một hình chữ nhật nhỏ hơn bên trong.
- Hãy làm bài theo thứ tự từ ngoài vào trong và từ trên xuống dưới.
- Nếu làm sai 1 bước, có thể bấm "**Lùi lại 1 bước**" để làm lại bước trước đó.
- Liệt kê các đỉnh theo thứ tự **A, B, C, ...**
- **k được khởi tạo = 1.**

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc





Áp dụng thuật toán Tarjan bằng cách duyệt đệ quy theo chiều sâu bắt đầu từ đỉnh C

Lùi lại 1 bước Số bước: 30

SCC(C)

1. Đánh số cho đỉnh C và đưa nó vào ngăn xếp S.

Gán $\text{num}[C] = \text{min_num}[C] = k = 1$; $k++$;

Đưa C vào S. Kết quả: S = C

Thực hiện đánh số và đưa vào Stack ☒ 1

2. Với các đỉnh kề v của C: B,E ☒ 2

2a. v = 'B' chưa duyệt ☒ 3

SCC(B)

1. Đánh số cho đỉnh B và đưa nó vào ngăn xếp S.

Gán $\text{num}[B] = \text{min_num}[B] = k = 2$; $k++$;

Đưa B vào S. Kết quả: S = C,B

Thực hiện đánh số và đưa vào Stack ☒ 4

2. Với các đỉnh kề v của B: D ☒ 5

2a. v = 'D' chưa duyệt ☒ 6

SCC(D)

1. Đánh số cho đỉnh D và đưa nó vào ngăn xếp S.

Gán $\text{num}[D] = \text{min_num}[D] = k = 3$; $k++$;

Đưa D vào S. Kết quả: S = C,B,D

Thực hiện đánh số và đưa vào Stack ☒ 7

2. Với các đỉnh kề v của D: E ☒ 8

2a. v = 'E' chưa duyệt ☒ 9

SCC(E)

1. Đánh số cho đỉnh E và đưa nó vào ngăn xếp S.

Gán $\text{num}[E] = \text{min_num}[E] = k = 4$; $k++$;

Đưa E vào S. Kết quả: S = C,B,D,E

Thực hiện đánh số và đưa vào Stack ☒ 10

2. Với các đỉnh kề v của E: A,D ☒ 11

2a. v = 'A' chưa duyệt ☒ 12

SCC(A)

1. Đánh số cho đỉnh A và đưa nó vào ngăn xếp S.

Gán $\text{num}[A] = \text{min_num}[A] = k = 5$; $k++$;

Đưa A vào S. Kết quả: S = C,B,D,E,A

Thực hiện đánh số và đưa vào Stack ☒ 13

2. Với các đỉnh kề v của A:

Xét ☒ 14

3. Kiểm tra num và min_num của A

$\text{num}[A] == \text{min_num}[A]$ ☒ $\text{num}[A] != \text{min_num}[A]$ ☒ 15

Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được A

Các đỉnh được lấy ra: A

Nội dung còn lại của ngăn xếp: C,B,D,E

Cập nhật ☒ 16

Cập nhật $\text{min_num}[E] = \min(\text{min_num}[E], \text{min_num}[A]) = 4$

Cập nhật ☒ 17

2b. v = 'D' duyệt rồi nhưng vẫn còn trên stack Cập nhật $\text{min_num}[u]$ ☒ 18

Cập nhật $\text{min_num}[E] = \min(\text{min_num}[E], \text{num}[D]) = 3$

Cập nhật ☒ 19

3. Kiểm tra num và min_num của E

$\text{num}[E] == \text{min_num}[E]$ ☒ $\text{num}[E] != \text{min_num}[E]$ ☒ 20

Cập nhật $\text{min_num}[D] = \min(\text{min_num}[D], \text{min_num}[E]) = 3$

Cập nhật ☒ 21

3. Kiểm tra num và min_num của D

$\text{num}[D] == \text{min_num}[D]$ ☒ $\text{num}[D] != \text{min_num}[D]$ ☒ 22

Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được D

Các đỉnh được lấy ra: E,D

Nội dung còn lại của ngăn xếp: C,B

Cập nhật ☒ 23

Cập nhật $\text{min_num}[B] = \min(\text{min_num}[B], \text{min_num}[D]) = 2$

Cập nhật ☒ 24

3. Kiểm tra num và min_num của B

$\text{num}[B] == \text{min_num}[B]$ ☒ $\text{num}[B] != \text{min_num}[B]$ ☒ 25

Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được B

Các đỉnh được lấy ra: B

Nội dung còn lại của ngăn xếp: C

Cập nhật ☒ 26

Cập nhật $\text{min_num}[C] = \min(\text{min_num}[C], \text{min_num}[B]) = 1$

Cập nhật ☒ 27

2c. v = 'E' duyệt rồi và không còn trên stack Bỏ qua ☒ 28

3. Kiểm tra num và min_num của C

$\text{num}[C] == \text{min_num}[C]$ ☒ $\text{num}[C] != \text{min_num}[C]$ ☒ 29

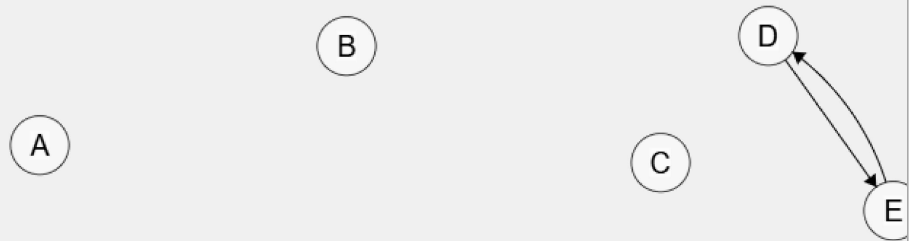
Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được C

Các đỉnh được lấy ra: C

Nội dung còn lại của ngăn xếp:

Cập nhật ☒ 30

Help Clear shift Delete Edit Undo Red Black



	Test	Got	
✓	Test tự động	1. Kiểm tra DFS - Bước 1. SCC(C): 1. Đánh số và đưa u vào ngăn xếp, với $u = C$ - Bước 2. SCC(C): 2. Xử lý các đỉnh kề của C - Bước 3. SCC(C): 2a. Duyệt B - Bước 4. SCC(B): 1. Đánh số và đưa u vào ngăn xếp, với $u = B$ - Bước 5. SCC(B): 2. Xử lý các đỉnh kề của B - Bước 6. SCC(B): 2a. Duyệt D - Bước 7. SCC(D): 1. Đánh số và đưa u vào ngăn xếp, với $u = D$ - Bước 8. SCC(D): 2. Xử lý các đỉnh kề của D - Bước 9. SCC(D): 2a. Duyệt E - Bước 10. SCC(E): 1. Đánh số và đưa u vào ngăn xếp, với $u = E$ - Bước 11. SCC(E): 2. Xử lý các đỉnh kề của E - Bước 12. SCC(E): 2a. Duyệt A - Bước 13. SCC(A): 1. Đánh số và đưa u vào ngăn xếp, với $u = A$ - Bước 14. SCC(A): 2. Xử lý các đỉnh kề của A - Bước 15. SCC(A): 3a. $id[u] == min_id[u]$ với $u = A$ - Bước 16. SCC(A): 3a. Tìm được SCC của A - Bước 17. SCC(E): 2a. Cập nhật min_id của A - Bước 18. SCC(E): 2b. Cập nhật min_id của D - Bước 19. SCC(E): 2b. Cập nhật min_id của D - Bước 20. SCC(E): 3b. $id[u] != min_id[u]$ với $u = E$ - Bước 21. SCC(D): 2a. Cập nhật min_id của E - Bước 22. SCC(D): 3a. $id[u] == min_id[u]$ với $u = D$ - Bước 23. SCC(D): 3a. Tìm được SCC của D - Bước 24. SCC(B): 2a. Cập nhật min_id của D - Bước 25. SCC(B): 3a. $id[u] == min_id[u]$ với $u = B$ - Bước 26. SCC(B): 3a. Tìm được SCC của B - Bước 27. SCC(C): 2a. Cập nhật min_id của B - Bước 28. SCC(C): 2c. Bỏ qua đỉnh E - Bước 29. SCC(C): 3a. $id[u] == min_id[u]$ với $u = C$ - Bước 30. SCC(C): 3a. Tìm được SCC của C 2. Kiểm tra các thành phần liên thông Các thành phần liên thông Ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.90/1.00**.

Mark 0.39 out of 1.00

Tính thời gian trễ nhất có thể bắt đầu công việc u: $T[u]$

- Hai công việc giả là α và β . Để gõ được ký hiệu này, gõ `\alpha` và `\beta` vào đỉnh tương ứng.

Bảng công việc

[illegible]

	Test	Got	
✓	1. Mô hình hoá về đồ thị (30%) 2. Tính thời gian (70%)	1. Kiểm tra đồ thị & xếp hạng [E] Số cung giữa hai đỉnh (C, G) không đúng. [E] Số cung giữa hai đỉnh (C, β) không đúng. [E] Số cung giữa hai đỉnh (F, G) không đúng. [I] Vị trí đỉnh α ok. [I] Vị trí đỉnh A ok. [I] Vị trí đỉnh B ok. [I] Vị trí đỉnh C ok. [I] Vị trí đỉnh D ok. [I] Vị trí đỉnh E ok. [I] Vị trí đỉnh F ok. [I] Vị trí đỉnh G ok. [I] Vị trí đỉnh β ok. Tổng (1): 9.91/10. 2. Kiểm tra thời gian 2a Kiểm tra d[u] [E] d[F] sai. 2b Kiểm tra t[u] [E] t[A] sai. [E] t[B] sai. [E] t[C] sai. [E] t[D] sai. [E] t[E] sai. [E] t[F] sai. [E] t[G] sai. [E] t[β] sai. 2c Kiểm tra T[u] [E] T[α] sai. [E] T[A] sai. [E] T[B] sai. [E] T[C] sai. [E] T[D] sai. [E] T[E] sai. [E] T[F] sai. [E] T[G] sai. [E] T[β] sai. 2d Kiểm tra công việc then chốt. [E] Công việc [α] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [B] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [D] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [E] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [F] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [G] sai. Công việc then chốt có t[u] = T[u]. [E] Công việc [β] sai. Công việc then chốt có t[u] = T[u]. Tổng (2): 1.67/12.	✓

Partially correct

Marks for this submission: 0.39/1.00. Accounting for previous tries, this gives **0.39/1.00**.

Question 7

Incorrect

Mark 0.00 out of 1.00

Cho đồ thị **vô hướng**, có trọng số không âm gồm **6** đỉnh và **8** cung (xem hình vẽ bên dưới).

Áp dụng [Thuật toán Moore - Dijkstra](#) để tìm (các) đường đi ngắn nhất từ đỉnh **1** đến các đỉnh khác.

Vẽ cây đường đi ngắn nhất.

Quy ước

- Với mỗi đỉnh u , lưu trữ thông tin $pi[u]$ và $p[u]$ theo mẫu: **$u(pi[u]/p[u])$** , ví dụ: 1(0/-), 2(3/1)
- Sử dụng **oo** (hai chữ 'o') để biểu diễn vô cùng
- Nếu $p[u]$ chưa biết, ghi '-'
- Nhấp double trên đỉnh để đánh dấu/bỏ đánh dấu nó
- Nếu có hai đỉnh có cùng π , chọn đỉnh có thứ tự nhỏ

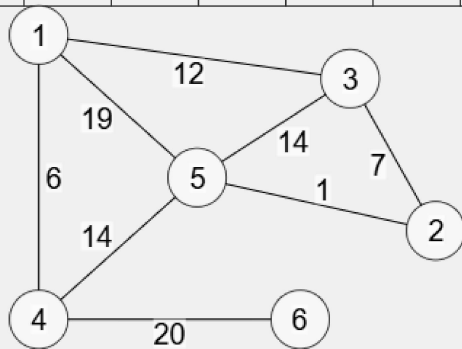
Answer: (penalty regime: 10, 20, ... %)

Reset answer

Given graph (Drag and drop to reorganize the graph)



Help Clear shift Delete Edit Undo Red Black

**1. Áp dụng thuật toán Moore - Dijkstra**

Init

Copy previous graph

Help Clear shift Delete Edit Undo Red Black

Bước 1

Copy previous graph

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Bước 2

Copy previous graph

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Bước 3

Copy previous graph

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Bước 4

Copy previous graph

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Bước 5

Copy previous graph

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

2. Vẽ cây đường đi ngắn nhất

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

	Test	Got	
✖	1. Thuật toán Moore - Dijkstra (80%) 2. Cây đường đi ngắn nhất (20%)	1. Kiểm tra áp dụng thuật toán Moore - Dijkstra + Bước khởi tạo: [E] Cây rỗng. Tổng (1): 0/18. 2. Kiểm tra cây đường đi ngắn nhất - [E] Vẽ cây sai: Cây rỗng. Tổng (2): 0/11.	✖

Incorrect

Marks for this submission: 0.00/1.00.

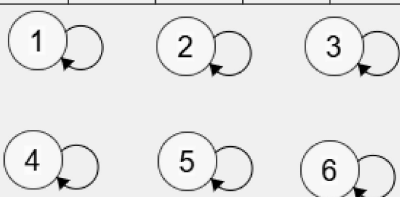
Mark 0.00 out of 1.00

[illegible]

	u	v	w	root_u	root_v	Thêm vào cây?
4						
5						
6						
7						
8						
9						
10						

Quản lý các BPLT

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------



Cây khung nhỏ nhất

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

	Test	Got	
✗	1. Thuật toán Kruskal (80%) 2. Quản lý các BPLT (10%) 3. Cây khung nhỏ nhất (10%)	1. Kiểm tra áp dụng thuật toán - [E] Bảng rỗng. Tổng (1): 0/20 Điểm: 0.00/10	✗

Incorrect

Marks for this submission: 0.00/1.00.

Question 9

Correct

Mark 1.00 out of 1.00

Cho đồ thị **có hướng** có trọng số gồm **6** đỉnh và **11** cung như bên như bên dưới.

Hãy áp dụng thuật toán Chu-Liu/Edmonds tìm cây khung có hướng nhỏ nhất của đồ thị trên.

Quy ước

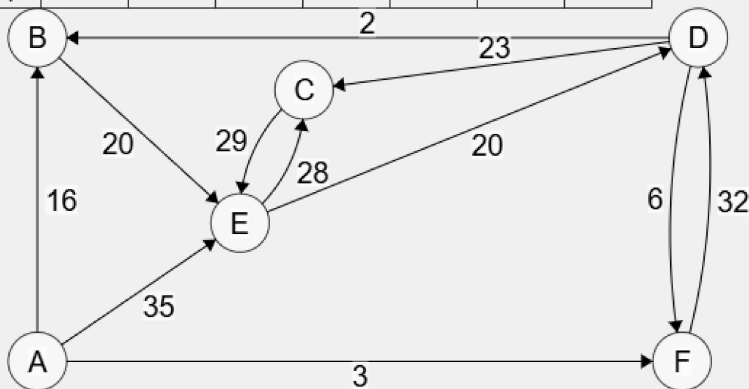
- Đặt tên các đỉnh mới bằng cách ghép tên các đỉnh cũ có trong chu trình. Ví dụ: nếu chu trình gồm các đỉnh B, C và E, thì đỉnh mới sau khi gộp sẽ có tên là **B,C,E**.
- Từ bước G1 trở đi, để dễ truy vết nên ghi nhãn của các cung theo dạng: **9-2 (A, C)** có nghĩa là trọng số mới = trọng số cũ (9) - trọng số của cung đi đến đỉnh trong chu trình (2), và (A, C) là cung trước khi co. Những phần ghi trong dấu ngoặc đơn là chú thích và sẽ được bỏ qua trong quá trình chấm.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc G0 (Dùng chuột để thay đổi vị trí của các đỉnh/cung)

Help Clear shift Delete Edit Undo Red Black



Áp dụng thuật toán Chu-Liu/Edmonds bằng cách vẽ các đồ thị vào các ô tương ứng bên dưới

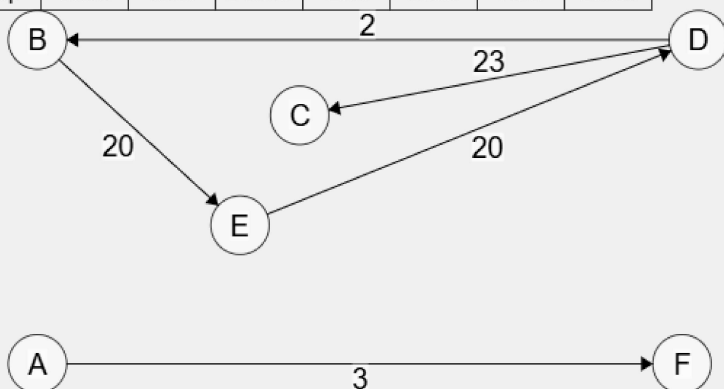
I. Pha co

Lập 0:

- Xây dựng đồ thị xấp xỉ H0: ☐ Copy đồ thị G0

Đồ thị xấp xỉ H0

Help Clear shift Delete Edit Undo Red Black

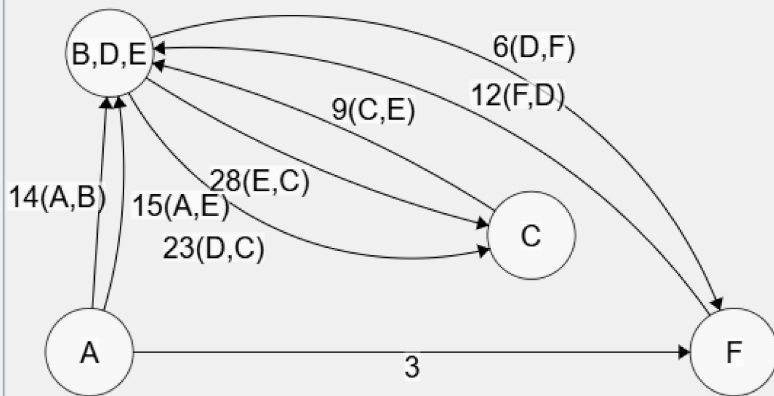


- Co đồ thị G0 thành G1: ☐ Copy đồ thị G0

Đồ thị G1

Help Clear shift Delete Edit Undo Red Black

Help Clear shift Delete Edit Undo Red Black



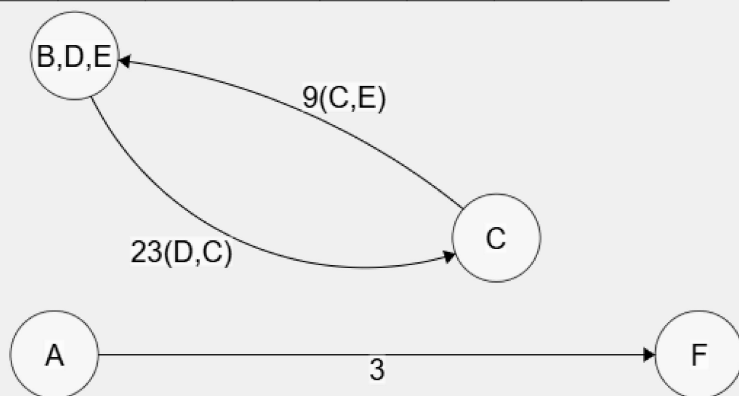
Lập 1:

- Xây dựng đồ thị xấp xỉ H1: ☐ Copy đồ thị G1

Đồ thị H1

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black

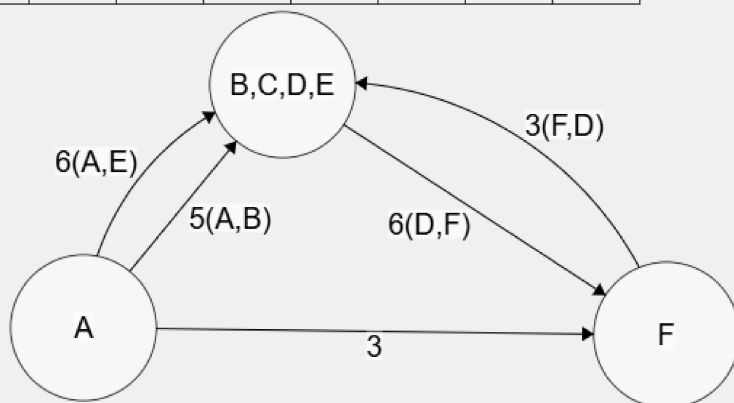


- Co đồ thị G1 thành G2: ☐ Copy đồ thị G1

Đồ thị G2

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black



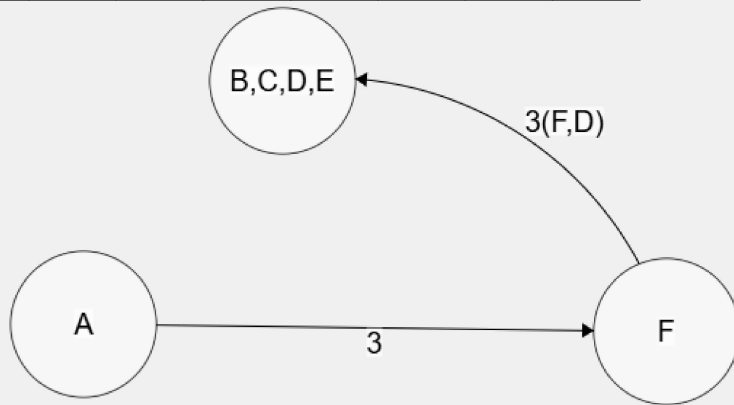
Lập 2:

- Xây dựng đồ thị xấp xỉ H2: ☐ Copy đồ thị G2

Đồ thị H2

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black



II. Pha giản

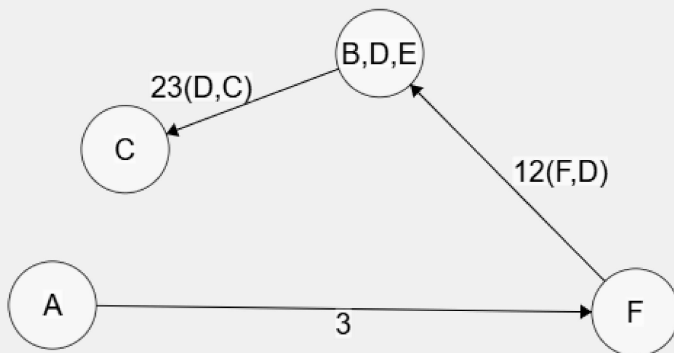
- T2 = H2

- Giãn T2 thành T1: ☐ Copy đồ thị T2

Cây khung T1



Help Clear shift Delete Edit Undo Red Black

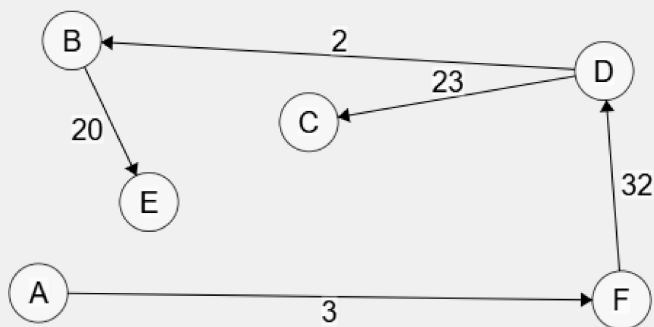


- Giãn T1 thành T0: ☐ Copy đồ thị T1

Cây khung T0



Help Clear shift Delete Edit Undo Red Black



	Test	Got	
✓	I. PHA CO II. PHA GIÃN	I. PHA CO Xây dựng đồ thị xấp xỉ H0: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị co G1: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị xấp xỉ H1: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị co G2: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị xấp xỉ H2: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 H2 không chứa chu trình => thoát vòng lặp, chuyển sang PHA GIÃN II. PHA GIÃN T2 = H2. Giãn cây T2 thành T1: - Tất cả các bước đều okie. - [I] Điểm: 1.0/1.0 Giãn cây T1 thành T0: - Tất cả các bước đều okie. - [I] Điểm: 1.0/1.0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 10

Correct

Mark 0.50 out of 1.00

Cho mạng với đỉnh phát $s = A$ và đỉnh thu $t = F$ được biểu diễn bằng đồ thị như bên dưới.

Hãy áp dụng thuật toán Ford - Fulkerson tìm luồng cực đại trong mạng. Thuật toán gồm 2 bước chính:

- Khởi tạo một luồng hợp lệ bất kỳ (thường là luồng có giá trị 0)
- Lập để tìm tăng luồng. Việc tìm đường tăng luồng được thực hiện bằng cách gán các đỉnh dùng **hàng đợi** (theo thuật toán Edmonds-Karp)

Sau bước gán nhãn, nếu tìm được đường tăng luồng, hãy vẽ lại mạng sau khi tăng luồng và tiếp tục. Ngược lại nếu không tìm được đường tăng luồng, thuật toán kết thúc, hãy cho biết lát cắt hẹp nhất (S, T) và giá trị luồng cực đại trong mạng.

Quy ước

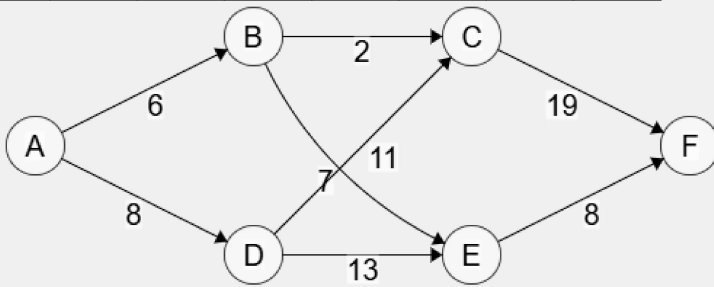
- Ghi nhãn các đỉnh theo mẫu: $(d[u], p[u], o[u])$ ví dụ: $(+, A, oo)$, $(-, D, 5)$
- Cột **Hàng đợi** ghi các đỉnh đang có trong hàng đợi, ngăn cách bằng dấu phẩy. Đầu hàng đợi (front) nằm bên trái. Ví dụ: B, D
- Ghi luồng trên cung theo mẫu: f/c hoặc f (ví dụ: $3/5$ hoặc 3) với f là luồng trên cung và c là khả năng thông qua của cung
- Khi xét đỉnh để gán nhãn, gán nhãn đỉnh liên quan đến **cung thuận trước, cung nghịch sau, đỉnh có thứ tự nhỏ trước, đỉnh có thứ tự lớn sau**

Answer: (penalty regime: 10, 20, ... %)

Reset answer

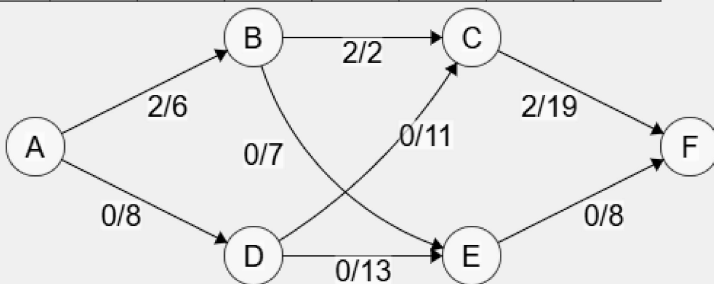
Mạng ban đầu

Help Clear shift Delete Edit Undo Red Black



I. Khởi tạo một luồng hợp lệ có giá trị không vượt quá 2

Help Clear shift Delete Edit Undo Red Black



II. Lập

Lần lặp 1

1.1 Gán nhãn các đỉnh dùng hàng đợi

	A	B	C	D	E	F	Hàng đợi
Khởi tạo	$(+, A, oo)$						A
1		$(+, A, 4)$		$(+, A, 8)$			B, D

	A	B	C	D	E	F	Hàng đợi
2					(+,B,4)		D,E
3			(+,D,8)				E,C
4						(+,E,4)	C,F
5							F

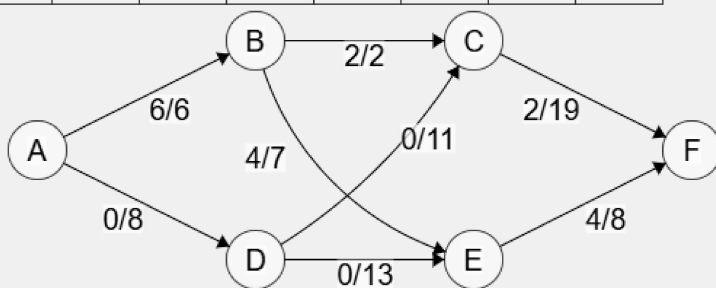
Add row Delete row

☒ Tìm được đường tăng luồng ☐ Không có đường tăng luồng

- Đường tăng luồng (augmenting path): A->B->E->F ví dụ: A -> C -> F
- Lượng luồng tăng thêm (bottle neck capacity): 4

1.2 Mạng sau khi tăng luồng ☐ Copy mạng ở bước trên

Help Clear shift Delete Edit Undo Red Black



Lần lặp 2

1.1 Gán nhãn các đỉnh dùng hàng đợi

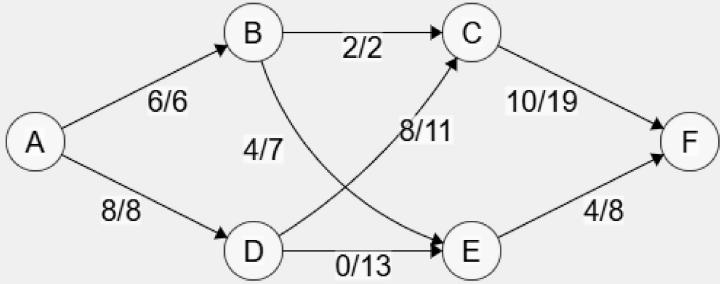
	A	B	C	D	E	F	Hàng đợi
Khởi tạo	(+,A,∞)						A
1				(+,A,8)			D
2			(+,D,8)		(+,D,8)		C,E
3		(-,C,2)				(+,C,8)	E,F,B
4							F
5							B

Add row Delete row

☒ Tìm được đường tăng luồng ☐ Không có đường tăng luồng

- Đường tăng luồng (augmenting path): A->D->C->F ví dụ: A -> C -> F
- Lượng luồng tăng thêm (bottle neck capacity): 8

1.2 Mạng sau khi tăng luồng ☐ Copy mạng ở bước trên



Lần lặp 3

1.1 Gán nhãn các đỉnh dùng hàng đợi

	A	B	C	D	E	F	Hàng đợi
Khởi tạo	(+,A,∞)						A
1							
2							
3							
4							
5							

Add rowDelete row

☒ Tìm được đường tăng luồng

☐ Không có đường tăng luồng

III. Kết quả

- Luồng cực đại: 14
- Lát cắt hẹp nhất (S, T) tách s và t (ngăn cách các đỉnh bằng dấu phẩy, ví dụ: A, C, D):
 - S = A
 - T = B,C,D,E,F

	Test	Got	
✓	I. Khởi tạo luồng (20%) II. Lập tìm đường tăng luồng (70%) III. Lát cắt hẹp nhất và giá trị luồng cực đại (10%) 3. Mạng sau khi tăng luồng (10%)	I. Khởi tạo luồng [I] Khởi tạo luồng okie. II. Lập Lần lặp 1 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. Lần lặp 2: [I] Nhãn của các đỉnh okie. Lần lặp 3: [I] Nhãn của các đỉnh okie. Lần lặp 4: [I] Nhãn của các đỉnh okie. 2. Đường tăng luồng và lượng luồng tăng thêm [I] Đường tăng luồng okie. [I] Lượng luồng tăng thêm okie. 3. Kiểm tra mạng sau khi tăng luồng Lần lặp 2 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. Lần lặp 2: [I] Nhãn của các đỉnh okie. Lần lặp 3: [I] Nhãn của các đỉnh okie. 2. Đường tăng luồng và lượng luồng tăng thêm [I] Đường tăng luồng okie. [I] Lượng luồng tăng thêm okie. 3. Kiểm tra mạng sau khi tăng luồng Lần lặp 3 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. III. Kết quả (lát cắt và luồng cực đại) [I] Lát cắt hẹp nhất và giá trị luồng cực đại okie.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.50/1.00**.

◀ News forum

Jump to...

